

E155

# Advanced Java

Armored Drill

8 concepts · 12 items · 4 transfer items

**What this is.** Every concept in this course's public description, re-asked with more disguise than a practice test uses. Each stem carries facts that do not decide it. Every wrong answer is a named trap.

**Source.** Scope from the course's public description in the WGU Institutional Catalog (September 2026); items written from the field's standard body of knowledge. No WGU assessment item, course material or proprietary content is used or reproduced. Not affiliated with or endorsed by Western Governors University.

Method © 2026 Erik Daniel Lindquist — Course Mastery Coach, armored drill. Items drafted with AI assistance under his direction.

# How to run it

Paper first. Pencil. One pass, no notes, a timer if you want the pressure.

1. **Before you read the options**, fill the strip under the item: the target word (what is actually being asked), and the one fact in the stem that decides it.
2. **Principle before arithmetic**. Name the rule, then compute.
3. **Camouflage**. Most of every stem does not matter. Ask: which single fact decides this?
4. **Inverse pairs are the bait**. If two options are opposites, one of them is the trap built for you.
5. **Mark sure, unsure or guess** honestly. Sure-and-wrong is the most valuable miss on the page — it is a wrong mental model.
6. **Absolutes** (always, never, guarantee) are usually wrong. And recognizing an answer is not the same as owning it.
7. Items with a **heavy black bar** on the left are **transfer items**: the same principle, reversed or solved for a different variable.
8. Stop at the STOP page. Then grade, fill the triage grid, and tally the traps you fell for. The biggest tally is what you drill next.

The strip, filled in, looks like this:

|                             |                                           |                        |                                  |
|-----------------------------|-------------------------------------------|------------------------|----------------------------------|
| <b>Target word</b><br>_____ | <b>The one fact that decides</b><br>_____ | <b>Answer</b><br>_____ | [ ] sure [ ] unsure [ ]<br>guess |
|-----------------------------|-------------------------------------------|------------------------|----------------------------------|

# Items

1.

A ticketing service built by a team of eight developers keeps a shared int field named sold. Two threads each run sold++ 50,000 times with no synchronization on a 16-core cloud server. The code passed review last sprint. What final value of sold is guaranteed?

- A. At most 16 updates lost, one per core.
- B. No exact value is guaranteed; it can be less than 100,000.
- C. Exactly 50,000.
- D. Exactly 100,000.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

2. [TRANSFER — reversed stem / different variable]

The developer moves the increment into a synchronized method on the same shared object and reruns two threads of 50,000 increments each. What is the final value?

- A. 50,000
- B. Less than 100,000, because synchronized only affects visibility.
- C. Unpredictable, because threads always race.
- D. 100,000

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

3.

A monitoring agent deployed across 40 containers has a boolean running flag. One thread sets it to false to stop a worker loop in another thread, but in testing the worker never stops. Logs go to a third-party dashboard. What is the smallest correct fix?

- A. Declare the flag static.
- B. Declare the flag volatile.
- C. Declare the flag final.
- D. Send fewer logs to the dashboard.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

**4.** [TRANSFER — reversed stem / different variable]

A counter shared by threads is declared volatile and updated with `count++`. Is it thread-safe?

- A.** Yes; volatile makes all operations on the field atomic.
- B.** No, because volatile fields cannot be incremented.
- C.** Yes, as long as the field is also private.
- D.** No; use `AtomicInteger` or synchronization for the increment.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

**5.**

An image-processing service that handles uploads for a photo-sharing startup creates a new Thread for every request. During a spike of 10,000 requests per minute, the service runs out of memory. The startup recently rebranded. What is the standard fix?

- A.** Create 10,000 threads at startup so they are ready.
- B.** Keep one thread per request but raise the heap size.
- C.** Submit tasks to a bounded `ExecutorService` thread pool and shut it down on exit.
- D.** Call `run()` instead of `start()` on each thread.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

**6.**

In a banking module that serves 2 million customers, Thread A locks account X and then waits to lock Y, while Thread B locks Y and then waits to lock X. Both hang. The module is written in Java 21. What prevents this?

- A.** Always acquire locks in one consistent order, such as by account ID.
- B.** Add more threads so one of them finishes.
- C.** Mark each transfer method synchronized on its own account object.
- D.** Upgrade to a newer Java version.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

**7.** [TRANSFER — reversed stem / different variable]

Two threads each need two locks. Which condition, if removed, makes deadlock between them impossible?

- A.** The use of any locks at all.
- B.** Circular wait.
- C.** Starvation.
- D.** A race condition.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

8.

A pricing service calls three independent supplier APIs that each take about 2 seconds. The developer submits the first call to an `ExecutorService` and immediately calls `get()` on its `Future`, then does the same for the second and third. The suppliers are located in three countries. About how long does the whole job take?

- A. About 3 seconds, one per country.
- B. About 2 seconds.
- C. Almost instantly, because `Futures` never block.
- D. About 6 seconds.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

9.

A Spring Boot order service for a furniture retailer creates its payment client with new `PaymentClient()` inside its constructor. Every unit test is slow because it calls the real payment sandbox. The retailer ships to 12 states. Which design change best fixes testability?

- A. Inject a payment interface through the constructor so tests can pass a mock.
- B. Make the payment client a static field.
- C. Always use field injection by reflection for every dependency.
- D. Move the new call into a private helper method in the same class.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

10. [TRANSFER — reversed stem / different variable]

In a Spring application, what is responsible for creating a bean and supplying it to the classes that need it?

- A. The JVM garbage collector.
- B. The database driver.
- C. The Spring IoC container (application context).
- D. The class that uses the bean.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

**11.**

A developer on a six-person team puts SQL queries, discount rules, and JSON formatting all inside one `@RestController` method. The app runs on Kubernetes. Code review asks for proper layering. Where should the discount rules live?

- A.** In a `@Service` class.
- B.** In the Kubernetes deployment configuration.
- C.** In the `@Repository`.
- D.** In the controller, since it receives the request.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

**12.**

An inventory API for a sporting-goods chain stores each user's shopping cart in the memory of the instance serving them. After moving to a cloud platform that autoscales from 2 to 10 instances behind a load balancer, users lose carts at random. The chain has 300 stores. What is the best fix?

- A.** Fix the instance count at 10.
- B.** Save carts to local files on each instance.
- C.** Make the instances stateless and keep carts in an external store such as a distributed cache or database.
- D.** Turn off autoscaling and run one larger server.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

# STOP

Answers and dissection follow. Finish every item and every strip first.

# Key and dissection

Struck-through text is the camouflage: facts that do not decide the item.

## 1. Answer B Concept 1

A ticketing service ~~built by a team of eight developers~~ keeps a shared int field named sold. Two threads each run sold++ 50,000 times with no synchronization ~~on a 16-core cloud server. The code passed review last sprint.~~ What final value of sold is guaranteed?

**Target word:** guaranteed final value **Decides:** sold++ is not atomic and nothing guards it.

|          |                   |                                                                                |
|----------|-------------------|--------------------------------------------------------------------------------|
| A        | Camouflage        | Core count does not cap lost updates.                                          |
| <b>B</b> | <b>Correct</b>    | Interleaved read-modify-write steps can overwrite each other and lose updates. |
| C        | Wrong stem number | One thread's count, not a guaranteed total.                                    |
| D        | Dropped element   | Drops the race condition; that total needs synchronization.                    |

## 2. Answer D Concept 1 · transfer

The developer moves the increment into a synchronized method on the same shared object and reruns two threads of 50,000 increments each. What is the final value?

**Target word:** result once the race is fixed **Decides:** Synchronization makes each increment mutually exclusive.

|          |                   |                                                                              |
|----------|-------------------|------------------------------------------------------------------------------|
| A        | Wrong stem number | One thread's share.                                                          |
| B        | Confusion set     | Describes volatile; synchronized gives both mutual exclusion and visibility. |
| C        | Absolute language | Proper locking removes the race.                                             |
| <b>D</b> | <b>Correct</b>    | Each increment runs alone under the lock, so none are lost.                  |

## 3. Answer B Concept 2

A monitoring agent ~~deployed across 40 containers~~ has a boolean running flag. One thread sets it to false to stop a worker loop in another thread, but in testing the worker never stops. ~~Logs go to a third-party dashboard.~~ What is the smallest correct fix?

**Target word:** minimal fix **Decides:** The worker may never see the other thread's write.

|          |                    |                                                                             |
|----------|--------------------|-----------------------------------------------------------------------------|
| A        | Adjacent category  | static changes ownership (class vs. instance), not cross-thread visibility. |
| <b>B</b> | <b>Correct</b>     | volatile makes the write visible to the worker thread.                      |
| C        | Inverted operation | final would stop the flag from ever being changed.                          |
| D        | Camouflage         | Logging does not decide whether a thread sees the write.                    |

## 4. Answer D Concept 2 · transfer

A counter shared by threads is declared volatile and updated with count++. Is it thread-safe?

**Target word:** does volatile make it safe **Decides:** volatile gives visibility, not atomicity.

|          |                   |                                                                      |
|----------|-------------------|----------------------------------------------------------------------|
| A        | Absolute language | Only single reads and writes are made visible, not compound updates. |
| B        | Confusion set     | They can be incremented; the problem is atomicity.                   |
| C        | Adjacent category | Access modifiers do not affect thread safety.                        |
| <b>D</b> | <b>Correct</b>    | count++ is still three steps that can interleave.                    |

## 5. Answer C Concept 3

An image-processing service ~~that handles uploads for a photo-sharing startup~~ creates a new Thread for every request. During a spike of 10,000 requests per minute, the service runs out of memory. ~~The startup recently rebranded.~~ What is the standard fix?

**Target word:** standard fix **Decides:** Unbounded thread creation per request.

|   |                   |                                                                       |
|---|-------------------|-----------------------------------------------------------------------|
| A | Wrong stem number | Reuses the request count as a pool size; far too many threads.        |
| B | Dropped element   | Drops the root cause; unbounded threads will still exhaust resources. |
| C | <b>Correct</b>    | A pool reuses a fixed number of threads and queues extra work.        |
| D | Confusion set     | run() executes on the caller's thread, removing concurrency.          |

## 6. Answer A Concept 4

In a banking module ~~that serves 2 million customers~~, Thread A locks account X and then waits to lock Y, while Thread B locks Y and then waits to lock X. Both hang. ~~The module is written in Java 21.~~ What prevents this?

**Target word:** prevention **Decides:** The two threads take the same locks in opposite order.

|   |                    |                                                                  |
|---|--------------------|------------------------------------------------------------------|
| A | <b>Correct</b>     | A fixed order removes the circular wait.                         |
| B | Inverted operation | More threads add contention; the stuck pair still waits forever. |
| C | Adjacent category  | Still locks two objects in varying order, so the cycle remains.  |
| D | Camouflage         | The JVM version does not fix lock ordering.                      |

## 7. Answer B Concept 4 · transfer

Two threads each need two locks. Which condition, if removed, makes deadlock between them impossible?

**Target word:** reverse: name the condition **Decides:** Deadlock requires a cycle of waiting.

|   |                   |                                                                              |
|---|-------------------|------------------------------------------------------------------------------|
| A | Absolute language | Locks are fine; the ordering is what creates the cycle.                      |
| B | <b>Correct</b>    | Without a cycle, some thread can always proceed.                             |
| C | Confusion set     | Starvation is a thread never getting a turn; it is not a deadlock condition. |
| D | Adjacent category | Races cause wrong results, not mutual waiting.                               |

## 8. Answer D Concept 5

A pricing service calls three independent supplier APIs that each take about 2 seconds. The developer submits the first call to an ExecutorService and immediately calls get() on its Future, then does the same for the second and third. ~~The suppliers are located in three countries.~~ About how long does the whole job take?

**Target word:** total time **Decides:** get() blocks before the next task is submitted.

|   |                   |                                                                                             |
|---|-------------------|---------------------------------------------------------------------------------------------|
| A | Camouflage        | Supplier location does not decide timing.                                                   |
| B | Dropped element   | Would be true only if all three were submitted before any get().                            |
| C | Absolute language | get() blocks until the result is ready.                                                     |
| D | <b>Correct</b>    | Each get() waits 2 seconds before the next call starts, so the calls run one after another. |

## 9. Answer A Concept 6

A Spring Boot order service ~~for a furniture retailer~~ creates its payment client with new PaymentClient() inside its constructor. Every unit test is slow because it calls the real payment sandbox. ~~The retailer ships to 12 states.~~ Which design change best fixes testability?

**Target word:** design fix **Decides:** The class builds its own dependency, so tests cannot swap it.

|   |                   |                                                                                                   |
|---|-------------------|---------------------------------------------------------------------------------------------------|
| A | <b>Correct</b>    | Constructor injection decouples the class and lets the container or a test supply the dependency. |
| B | Adjacent category | Still hard-wired; static state is even harder to replace in tests.                                |
| C | Absolute language | Constructor injection is preferred; field injection hides dependencies.                           |
| D | Dropped element   | The class still creates its own dependency.                                                       |

## 10. Answer C Concept 6 · transfer

In a Spring application, what is responsible for creating a bean and supplying it to the classes that need it?

**Target word:** who does the wiring **Decides:** Inversion of control.

|   |                    |                                                               |
|---|--------------------|---------------------------------------------------------------|
| A | Confusion set      | The garbage collector frees memory; it does not wire objects. |
| B | Adjacent category  | Drivers connect to databases; they do not manage beans.       |
| C | <b>Correct</b>     | The container manages bean creation and injection.            |
| D | Inverted operation | That is the opposite of inversion of control.                 |

## 11. Answer A Concept 7

A developer ~~on a six person team~~ puts SQL queries, discount rules, and JSON formatting all inside one @RestController method. ~~The app runs on Kubernetes.~~ Code review asks for proper layering. Where should the discount rules live?

**Target word:** correct layer for business rules **Decides:** Discount rules are business logic.

|   |                   |                                                                                     |
|---|-------------------|-------------------------------------------------------------------------------------|
| A | <b>Correct</b>    | Services hold business rules; controllers handle HTTP and repositories handle data. |
| B | Camouflage        | Deployment settings are not business logic.                                         |
| C | Adjacent category | Repositories handle data access, not pricing rules.                                 |
| D | Confusion set     | Receiving the request is the controller's whole job; rules belong below it.         |

## 12. Answer C Concept 8

An inventory API for a sporting-goods chain stores each user's shopping cart in the memory of the instance serving them. After moving to a cloud platform that autoscales from 2 to 10 instances behind a load balancer, users lose carts at random. ~~The chain has 300 stores.~~ What is the best fix?

**Target word:** fix for scaling **Decides:** Requests land on different instances that do not share memory.

|   |                    |                                                                 |
|---|--------------------|-----------------------------------------------------------------|
| A | Wrong stem number  | More fixed instances still do not share cart memory.            |
| B | Dropped element    | Still ties state to one instance.                               |
| C | <b>Correct</b>     | Any instance can then serve any request, so scaling is safe.    |
| D | Inverted operation | Scaling up instead of out removes resilience and limits growth. |

## Triage grid

Write each item number in the box it belongs to.

|                       | Right                                                 | Wrong                                                                                 |
|-----------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Sure</b>           | Owned.                                                | <b>Concept inversion — top priority.</b> A wrong mental model; it survives rewording. |
| <b>Unsure / guess</b> | Recognition, not mastery. Re-drill with the transfer. | Not learned yet.                                                                      |

## Trap tally

For every miss, find the letter you chose in the key and tally its trap. The biggest number is the pattern to drill.

| Trap               | What it does                                                 | In this drill | Your misses |
|--------------------|--------------------------------------------------------------|---------------|-------------|
| Dropped element    | Leaves out a required term or step.                          | 5             |             |
| Wrong stem number  | A figure copied from the prompt that is not the answer.      | 4             |             |
| Adjacent category  | The right idea in the wrong bucket.                          | 7             |             |
| Inverted operation | Direction flipped: add/subtract, cause/effect, higher/lower. | 4             |             |
| Absolute language  | Always, never, guarantee — where judgment applies.           | 5             |             |
| Confusion set      | Every option is real; the test is exact role-matching.       | 6             |             |
| Camouflage         | Answers a stem fact that was never the question.             | 5             |             |

# Anchor rows — the concept list

One row per concept: the rule to lock in, the trap that catches people, and the tell in the question that signals it.

| # | Concept                                     | Rule                                                                                                                                                           | Trap                                                           | Tell                                                                                |
|---|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1 | <b>Race conditions on shared state</b>      | count++ is a read-modify-write; without synchronization, concurrent updates can be lost.                                                                       | Assuming a one-line increment is atomic.                       | Several threads updating the same field with no lock.                               |
| 2 | <b>volatile vs. atomicity</b>               | volatile guarantees visibility of writes across threads, not atomic compound operations.                                                                       | Using volatile to make count++ safe.                           | A flag one thread sets and another thread never sees change, or a volatile counter. |
| 3 | <b>Executors and thread pools</b>           | Submit tasks to a bounded ExecutorService instead of creating a thread per task, and shut it down when done.                                                   | new Thread() per request, or calling run() instead of start(). | Load spikes cause memory exhaustion from too many threads.                          |
| 4 | <b>Deadlock</b>                             | Deadlock needs circular wait; acquiring locks in one consistent global order prevents it.                                                                      | Adding threads or upgrading the JVM to fix a lock-order bug.   | Two threads each hold one lock and wait for the other's.                            |
| 5 | <b>Futures block on get()</b>               | Future.get() blocks until the result is ready; submit all independent tasks first, then collect results.                                                       | Believing work is parallel just because an executor is used.   | get() called right after each submit.                                               |
| 6 | <b>Dependency injection and IoC</b>         | The framework container creates beans and injects dependencies, usually through the constructor, so classes depend on interfaces and can be tested with mocks. | Creating collaborators with new inside the class.              | Hard-to-test code that calls new on its own dependencies.                           |
| 7 | <b>Layered architecture</b>                 | Controllers handle HTTP, services hold business rules, repositories handle data access.                                                                        | Putting business logic in the controller or repository.        | One method mixing SQL, rules, and response formatting.                              |
| 8 | <b>Stateless services for cloud scaling</b> | Keep instances stateless and put session state in an external store so any instance can serve any request and autoscaling works.                               | Scaling up one big server or pinning users to instances.       | Data loss or random errors after adding instances behind a load balancer.           |

**Course scope (public description):** Advanced Java builds on object-oriented programming expertise by emphasizing multithreaded development and the use of Java frameworks to create scalable, maintainable applications. Learners will apply advanced Java techniques to design and implement software solutions that meet organizational and business requirements. The course also introduces deployment strategies using modern cloud services, enabling learners to deliver and manage applications in distributed environments. This course requires intermediate proficiency in Java and object-oriented programming.